
Lecture 02

Conditions, Loops and Methods

Basics of Internet Technologies 2

From a linear program to an interactive console app

Learning Objectives

- Use boolean values to represent yes/no facts.
- Compare values with `==`, `!=`, `>`, `<`, `>=`, and `<=`.
- Combine conditions with `&&`, `||`, and `!`.
- Use `if`, `else if`, `else`, and `switch`.
- Repeat code with `while`, `for`, and `foreach`.
- Build a simple menu loop.
- Split code into methods with parameters and return values.
- Apply simple validation to console input.

Connection to Lecture 01

- Lecture 01 created a linear program: read values, convert them, calculate, print, end.
- That is useful, but real applications need choices and repetition.
- Today the program can respond to commands and keep running.
- We still avoid classes, files, JSON, users, databases, and web APIs.

Lesson Timing Plan

- 10 min - recap and problem with linear programs.
- 15 min - bool, comparisons, and logical operators.
- 20 min - if, else if, else, switch, and validation decisions.
- 20 min - while, for, foreach, and common loop mistakes.
- 25 min - methods, parameters, return values, void, and scope.
- 20 min - live coding the expense menu.
- 10 min - student increment, exercises, questions.

Problem: The Program Ends Too Quickly

```
Expense name: Coffee
```

```
Amount: 4.50
```

```
You spent 4.50 EUR on Coffee.
```

```
Program ends.
```

- The user can add only one item.
- There is no command choice.
- There is no way to show the result again.
- Invalid input is difficult to recover from.

Target Shape for Today

```
while application is running
{
    show menu
    read command
    execute selected operation
}
```

- This is the base shape of many console applications.
- The command controls what path the program takes.
- The loop keeps the application alive.

Boolean Values

```
bool isRunning = true;  
bool hasExpense = false;  
bool amountIsValid = true;
```

- bool stores true or false.
- Conditions use bool values to choose a path.
- A bool variable should usually sound like a yes/no fact.

Comparison Operators

- `==` checks whether two values are equal.
- `!=` checks whether two values are not equal.
- `>` checks greater than.
- `<` checks less than.
- `>=` checks greater than or equal.
- `<=` checks less than or equal.
- Every comparison produces true or false.

Comparison Example

```
int balance = 100;  
int withdrawal = 40;  
  
bool hasEnoughMoney = balance >= withdrawal;  
bool tooLarge = withdrawal > balance;
```

- The values are numbers, but the comparison result is bool.
- These bool results can be used in if statements.
- Readable bool names make conditions easier to understand.

Logical Operators

```
bool canWithdraw = hasEnoughMoney && cardIsActive;  
bool needsHelp = !cardIsActive || withdrawal > 500;
```

- && means both sides must be true.
- || means at least one side must be true.
- ! reverses true to false or false to true.
- Use parentheses when a condition becomes difficult to read.

Truth Table: AND

Expression

- true && true
- true && false
- false && true
- false && false

Result

- true
- false
- false
- false

Truth Table: OR

Expression

- `true || true`
- `true || false`
- `false || true`
- `false || false`

Result

- `true`
- `true`
- `true`
- `false`

if Statement

```
if (amount > 0)
{
    Console.WriteLine("Amount accepted.");
}
```

- The condition is inside parentheses.
- The code block runs only when the condition is true.
- Braces make the controlled block clear.

if and else

```
if (amount > 0)
{
    Console.WriteLine("Amount accepted.");
}
else
{
    Console.WriteLine("Amount must be positive.");
}
```

- else handles the alternative path.
- Only one of the two blocks runs.
- This is the basic shape of validation feedback.

else if

```
if (amount <= 0)
{
    Console.WriteLine("Amount must be positive.");
}
else if (amount > 1000)
{
    Console.WriteLine("Large amount. Check again.");
}
else
{
    Console.WriteLine("Amount accepted.");
}
```

- else if checks another condition only if the previous condition was false.
- The order of conditions matters.
- Use it when one decision has several possible outcomes.

Nested if

```
if (amountIsNumber)
{
    if (amount > 0)
    {
        Console.WriteLine("Accepted");
    }
}
```

- A nested if is an if inside another if.
- It is sometimes useful, but too much nesting becomes hard to read.
- Later we will learn patterns that reduce deep nesting.

switch

```
switch (command)
{
    case "1":
        Console.WriteLine("Add expense");
        break;
    case "2":
        Console.WriteLine("Show expense");
        break;
    default:
        Console.WriteLine("Unknown command");
        break;
}
```

- switch is useful when one value is compared with several fixed cases.
- Each case should normally end with break.
- default handles unsupported values.

Choosing if or switch

Use if

- Complex conditions
- Ranges such as amount > 100
- Several variables
- Validation logic

Use switch

- One command value
- Fixed options
- Menu choices
- Known labels

while Loop

```
while (isRunning)
{
    Console.WriteLine("Menu");
}
```

- while repeats while the condition is true.
- If the condition never becomes false, the loop does not stop.
- Menu applications often use while with an isRunning variable.

Stopping a while Loop

```
bool isRunning = true;

while (isRunning)
{
    string command = Console.ReadLine();

    if (command == "3")
    {
        isRunning = false;
    }
}
```

- The loop condition is checked before each iteration.
- Changing isRunning to false lets the program leave the loop.
- This is clearer for beginners than clever loop tricks.

for Loop

```
for (int index = 0; index < 3; index++)  
{  
    Console.WriteLine(index);  
}
```

- A for loop is useful when the number of repetitions is known.
- It has initialization, condition, and update parts.
- It is common when working with array indexes.

foreach Loop

```
string[] commands = { "add", "show", "exit" };  
  
foreach (string command in commands)  
{  
    Console.WriteLine(command);  
}
```

- foreach visits each value in a collection or array.
- It is easier when the index is not needed.
- Lecture 03 will use foreach with List<T>.

Loop Comparison

Loop

- while
- for
- foreach

Best Use

- Repeat until a condition changes
- Repeat with an index or fixed count
- Read each item in a collection

Common Loop Mistakes

- Forgetting to change the condition in a while loop.
- Using `<=` instead of `<` with array indexes.
- Changing a loop variable in two places.
- Putting input outside the loop when it must happen every iteration.
- Making one loop do too many unrelated jobs.

Live Demo Menu

```
1. Add expense
2. Show current expense
3. Exit
```

- The demo stores one current expense.
- The user can repeat commands.
- Validation rejects empty names and invalid amounts.
- This is still intentionally simple: no collections and no files yet.

Demo State Variables

```
string expenseName = "";  
decimal expenseAmount = 0m;  
bool hasExpense = false;  
bool isRunning = true;
```

- State means values the program remembers while running.
- hasExpense tells whether showing an expense makes sense.
- isRunning controls the menu loop.

Reading a Command

```
Console.Write("Choose command: " );  
string command = Console.ReadLine();
```

- The command arrives as text.
- For a beginner menu, text commands such as "1" are simple and visible.
- switch can decide what each command means.

Validating Input

```
bool amountIsValid = decimal.TryParse(  
    amountText,  
    CultureInfo.InvariantCulture,  
    out decimal enteredAmount);
```

- TryParse prevents the program from crashing on invalid input.
- The bool result tells whether conversion succeeded.
- The converted value is available after successful conversion.

Validation Conditions

```
if (enteredName == "")
{
    Console.WriteLine("Expense name cannot be empty.");
}
else if (!amountIsValid)
{
    Console.WriteLine("Amount must be a number.");
}
else if (enteredAmount <= 0)
{
    Console.WriteLine("Amount must be greater than zero.");
}
```

- Each invalid case gets a clear message.
- The order avoids using enteredAmount before checking conversion.
- This is a first step toward reliable backend validation.

Methods: Why They Exist

- A method gives a name to a block of code.
- A method can receive input through parameters.
- A method can return a result.
- Methods reduce repetition and make Program.cs easier to read.
- Today we use methods inside one file. Later, classes will organize methods into objects.

void Method

```
void PrintSeparator()  
{  
    Console.WriteLine("-----");  
}
```

- void means the method does not return a value.
- This method performs an action: printing.
- Call it by writing PrintSeparator();

Method with Parameters

```
void PrintExpense(string name, decimal amount)
{
    Console.WriteLine(name + ": " + amount + " EUR");
}
```

- Parameters are values the caller passes into the method.
- Inside the method, parameters behave like local variables.
- Different calls can pass different values.

Method with Return Value

```
decimal CalculateTotal(decimal price, int quantity)
{
    return price * quantity;
}
```

- The return type appears before the method name.
- return sends a value back to the caller.
- A calculation method should usually return a value instead of printing directly.

Boolean Method for Validation

```
bool IsPositiveAmount(decimal amount)
{
    return amount > 0;
}
```

- A bool method is useful for a yes/no question.
- The method name should read like a condition.
- This makes if statements easier to read.

Variable Scope

```
int outerNumber = 10;

if (outerNumber > 0)
{
    int innerNumber = 5;
}

Console.WriteLine(outerNumber);
```

- A variable exists inside the block where it is declared.
- innerNumber exists only inside the if block.
- Scope prevents unrelated code from using temporary values by accident.

Parameters vs Local Variables

Parameters

- Declared in method parentheses
- Provided by the caller
- Represent input to a method

Local Variables

- Declared inside a block
- Created while that block runs
- Used for temporary work

Returning vs Printing

Return

- Gives a value back
- Good for calculations
- Can be reused by other code

Print

- Shows text to the user
- Good for console UI
- Harder to reuse for calculations

Student Project Increment

- Add a menu loop to the project from Lecture 01.
- Create at least three commands.
- Use if/else or switch to choose commands.
- Use while to keep the application running.
- Move repeated code into methods.
- Add simple validation for empty text and invalid numbers.
- Keep the application single-user for now.

Student Example Commands

Project

- Expense Tracker
- Task Manager
- ATM Simulator
- Veterinary Clinic

Commands

- Add expense, show current expense, exit
- Add task, show task, exit
- Deposit, show balance, exit
- Add pet, show current pet, exit

What Not To Add Yet

- Do not add JSON files yet.
- Do not add classes yet.
- Do not add users or login yet.
- Do not add databases yet.
- Do not build a Web API yet.
- The goal is to master decisions, repetition, methods, and validation in a console program.

Common Mistakes Today

- Writing all menu logic in one huge block without methods.
- Forgetting break in switch cases.
- Creating an infinite loop by never changing isRunning.
- Parsing numbers before checking whether conversion succeeded.
- Using a method that prints when a method should return a value.
- Making Lecture 02 more complex than the student can explain.

Exercises

- Add one new command to the demo menu.
- Create a method that prints the menu.
- Create a method that validates a positive decimal amount.
- Rewrite one if/else decision as a switch if it naturally fits.
- Add a for loop that prints numbers from 1 to 5.
- Add a foreach loop over an array of command names.

Review Questions

- What is the difference between = and ==?
- When should we use &&?
- When is while better than for?
- When is foreach simpler than for?
- What does void mean?
- What is a parameter?
- What is a return value?
- Why does variable scope matter?

Summary

- Conditions let a program choose between paths.
- Loops let a program repeat work.
- switch is useful for simple menu commands.
- while is useful for application loops.
- Methods give names to actions and calculations.
- Validation is the first step toward reliable backend behaviour.
- The student project now becomes interactive, but still intentionally simple.